8 puzzle

```
import java.util.*;

public class EightPuzzleMatrix {

    //      Goal test
    public static boolean goalTest(int[][] state, int[][] goal) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (state[i][j] != goal[i][j])
                    return false;
            }
        }
        return true;
    }

    //      Find position of '0' (blank)
    public static int[] findZero(int[][] state) {
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (state[i][j] == 0)
                    return new int[]{i, j};
            }
        }
        return null;
    }

    //      Deep copy of matrix
    public static int[][] copyMatrix(int[][] mat) {
        int[][] newMat = new int[3][3];
        for (int i = 0; i < 3; i++) {
            newMat[i] = mat[i].clone();
        }
        return newMat;
    }

    //      moveGen(): Generate all valid next states
    public static List<int[][]> moveGen(int[][] state) {
        List<int[][]> neighbors = new ArrayList<>();
        int[] pos = findZero(state);
        int row = pos[0], col = pos[1];
```

```
    int[][] moves = {
        {row - 1, col}, // up
        {row + 1, col}, // down
        {row, col - 1}, // left
        {row, col + 1}  // right
    };

    for (int[] move : moves) {
        int newR = move[0], newC = move[1];
        if (newR >= 0 && newR < 3 && newC >= 0 && newC < 3) {
            int[][] newState = copyMatrix(state);
            newState[row][col] = newState[newR][newC];
            newState[newR][newC] = 0;
            neighbors.add(newState);
        }
    }
    return neighbors;
}

//      BFS using Queue
public static List<int[][]> bfs(int[][] start, int[][] goal) {
    Queue<int[][]> queue = new LinkedList<>();
    Set<String> visited = new HashSet<>();
    Map<String, int[][]> parent = new HashMap<>();

    queue.add(start);
    visited.add(Arrays.deepToString(start));
    parent.put(Arrays.deepToString(start), null);

    while (!queue.isEmpty()) {
        int[][] current = queue.poll();

        if (goalTest(current, goal)) {
            return constructPath(parent, current);
        }

        for (int[][] next : moveGen(current)) {
            String key = Arrays.deepToString(next);
            if (!visited.contains(key)) {
                visited.add(key);
                parent.put(key, current);
                queue.add(next);
            }
        }
    }
    return null;
}

//      Construct path using parent map
public static List<int[][]> constructPath(Map<String, int[][]> parent, int[][] goal) {
    List<int[][]> path = new ArrayList<>();
    int[][] current = goal;
```

```
        while (current != null) {
            path.add(current);
            current = parent.get(Arrays.deepToString(current));
        }
        Collections.reverse(path);
        return path;
    }

    //      Print matrix
    public static void printState(int[][] state) {
        for (int[] row : state) {
            for (int val : row) {
                System.out.print((val == 0 ? "_" : val) + " ");
            }
            System.out.println();
        }
        System.out.println();
    }

    //      Main
    public static void main(String[] args) {
        int[][] start = {
            {1, 2, 3},
            {4, 0, 5},
            {6, 7, 8}
        };

        int[][] goal = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 0}
        };

        List<int[][]> solution = bfs(start, goal);

        if (solution != null) {
            System.out.println("Solution found in " + (solution.size() - 1) + " moves:");
            for (int[][] step : solution) {
                printState(step);
            }
        } else {
            System.out.println("No solution found.");
        }
    }
}
```

word ladder

```
import java.util.*;

public class WordLadder {

    //      goalTest(): Check if word is equal to goal
    public static boolean goalTest(String word, String goal) {
        return word.equals(goal);
    }

    //      moveGen(): Generate all valid words by changing one letter at a time
    public static List<String> moveGen(String word, Set<String> dictionary) {
        List<String> neighbors = new ArrayList<>();
        char[] wordArr = word.toCharArray();

        for (int i = 0; i < wordArr.length; i++) {
            char original = wordArr[i];

            for (char c = 'a'; c <= 'z'; c++) {
                if (c == original) continue;

                wordArr[i] = c;
                String newWord = new String(wordArr);

                if (dictionary.contains(newWord)) {
                    neighbors.add(newWord);
                }
            }
            wordArr[i] = original; // Restore original letter
        }
        return neighbors;
    }

    //      BFS to find shortest path
    public static List<String> bfs(String start, String goal, Set<String> dictionary) {
        Queue<String> queue = new LinkedList<>();
        Set<String> visited = new HashSet<>();
        Map<String, String> parent = new HashMap<>();

        queue.add(start);
        visited.add(start);
        parent.put(start, null);

        while (!queue.isEmpty()) {
            String current = queue.poll();
```

```
            if (goalTest(current, goal)) {
                return constructPath(parent, goal);
            }

            for (String next : moveGen(current, dictionary)) {
                if (!visited.contains(next)) {
                    visited.add(next);
                    parent.put(next, current);
                    queue.add(next);
                }
            }
        }
        return null; // No path found
    }

    //      Construct final path from parent map
    public static List<String> constructPath(Map<String, String> parent, String goal) {
        List<String> path = new ArrayList<>();
        String current = goal;

        while (current != null) {
            path.add(current);
            current = parent.get(current);
        }
        Collections.reverse(path);
        return path;
    }

    public static void main(String[] args) {
        String start = "hit";
        String goal = "cog";

        // Dictionary (only valid words allowed in path)
        Set<String> dictionary = new HashSet<>(Arrays.asList(
            "hot", "dot", "dog", "lot", "log", "cog", "hit"
        ));

        List<String> solution = bfs(start, goal, dictionary);

        if (solution != null) {
            System.out.println("Word Ladder Found in " + (solution.size() - 1) + " steps:");
            for (String word : solution) {
                System.out.println(word);
            }
        } else {
            System.out.println("No transformation found.");
        }
    }
}
```

BFS

```
import java.util.*;

public class SimpleBFS {
    public static void main(String[] args) {
        // Create the graph using adjacency list
        Map<String, List<String>> graph = new HashMap<>();

        graph.put("A", Arrays.asList("B", "C"));
        graph.put("B", Arrays.asList("A", "D"));
        graph.put("C", Arrays.asList("A", "E"));
        graph.put("D", Arrays.asList("B"));
        graph.put("E", Arrays.asList("C"));

        bfs(graph, "A");  // Start BFS from node A
    }

    public static void bfs(Map<String, List<String>> graph, String start) {
        Queue<String> queue = new LinkedList<>();
        Set<String> visited = new HashSet<>();

        queue.add(start);
        visited.add(start);

        System.out.println("BFS Traversal Starting from " + start + ":");

        while (!queue.isEmpty()) {
            String current = queue.poll();
            System.out.print(current + " ");

            for (String neighbor : graph.get(current)) {
                if (!visited.contains(neighbor)) {
                    visited.add(neighbor);
                    queue.add(neighbor);
                }
```

```
        }
    }
  }
}
```

dfs

```
import java.util.*;

public class SimpleDFS {
   public static void main(String[] args) {
      // Creating the graph using adjacency list
      Map<String, List<String>> graph = new HashMap<>();

      graph.put("A", Arrays.asList("B", "C"));
      graph.put("B", Arrays.asList("A", "D"));
      graph.put("C", Arrays.asList("A", "E"));
      graph.put("D", Arrays.asList("B"));
      graph.put("E", Arrays.asList("C"));

      Set<String> visited = new HashSet<>();

      System.out.print("DFS Traversal Starting from A: ");
      dfs(graph, "A", visited);
   }

   public static void dfs(Map<String, List<String>> graph, String current,
Set<String> visited) {
      visited.add(current);
      System.out.print(current + " ");

      for (String neighbor : graph.get(current)) {
         if (!visited.contains(neighbor)) {
            dfs(graph, neighbor, visited);
         }
      }
   }
}
```

```
import java.util.*;

public class DFSUsingStack {
    public static void main(String[] args) {
        // Create graph using adjacency list
        Map<String, List<String>> graph = new HashMap<>();

        graph.put("A", Arrays.asList("B", "C"));
        graph.put("B", Arrays.asList("A", "D"));
        graph.put("C", Arrays.asList("A", "E"));
        graph.put("D", Arrays.asList("B"));
        graph.put("E", Arrays.asList("C"));

        System.out.print("DFS Traversal using Stack starting from A: ");
        dfsUsingStack(graph, "A");
    }

    public static void dfsUsingStack(Map<String, List<String>> graph, String
start) {
        Stack<String> stack = new Stack<>();
        Set<String> visited = new HashSet<>();

        stack.push(start);

        while (!stack.isEmpty()) {
            String current = stack.pop();

            if (!visited.contains(current)) {
                System.out.print(current + " ");
                visited.add(current);

                // Push neighbors in reverse order to process leftmost first
                List<String> neighbors = graph.get(current);
                for (int i = neighbors.size() - 1; i >= 0; i--) {
                    if (!visited.contains(neighbors.get(i))) {
                        stack.push(neighbors.get(i));
                    }
                }
            }
        }
    }
}
```

binary matrix

```
import java.util.*;

public class BinaryMatrixShortestPath8Dir {

    // 8 directions: up, down, left, right + 4 diagonals
    private static final int[] rowDir = {-1, -1, -1, 0, 0, 1, 1, 1};
    private static final int[] colDir = {-1, 0, 1, -1, 1, -1, 0, 1};

    // goalTest: check if reached bottom-right
    public static boolean goalTest(int r, int c, int n) {
        return r == n - 1 && c == n - 1;
    }

    // moveGen: generate valid neighbors (only 0 and within grid)
    public static List<int[]> moveGen(int[][] grid, int r, int c, boolean[][] visited)
{
        List<int[]> neighbors = new ArrayList<>();
        int n = grid.length;

        for (int i = 0; i < 8; i++) {
            int newR = r + rowDir[i];
            int newC = c + colDir[i];

            if (newR >= 0 && newR < n && newC >= 0 && newC < n
                    && grid[newR][newC] == 0 && !visited[newR][newC]) {
                neighbors.add(new int[]{newR, newC});
            }
        }
        return neighbors;
    }

    // BFS to find shortest path
    public static List<int[]> bfs(int[][] grid) {
        int n = grid.length;
```

```
        if (grid[0][0] != 0 || grid[n-1][n-1] != 0) return null;

        boolean[][] visited = new boolean[n][n];
        Queue<int[]> queue = new LinkedList<>();
        Map<String, int[]> parent = new HashMap<>();

        queue.add(new int[]{0, 0});
        visited[0][0] = true;
        parent.put("0,0", null);

        while (!queue.isEmpty()) {
            int[] current = queue.poll();
            int r = current[0], c = current[1];

            if (goalTest(r, c, n)) {
                return constructPath(parent, current);
            }

            for (int[] next : moveGen(grid, r, c, visited)) {
                visited[next[0]][next[1]] = true;
                queue.add(next);
                parent.put(next[0] + "," + next[1], current);
            }
        }
        return null; // no path
    }

    // Reconstruct path from parent map
    public static List<int[]> constructPath(Map<String, int[]> parent, int[] goal)
{
        List<int[]> path = new ArrayList<>();
        int[] current = goal;

        while (current != null) {
            path.add(current);
            current = parent.get(current[0] + "," + current[1]);
        }
        Collections.reverse(path);
        return path;
    }

    // Print path coordinates
```

```
    public static void printPath(List<int[]> path) {
        for (int[] pos : path) {
            System.out.print("(" + pos[0] + "," + pos[1] + ") ");
        }
        System.out.println();
    }

    public static void main(String[] args) {
        int[][] grid = {
            {0, 0, 1, 0},
            {1, 0, 1, 0},
            {0, 0, 0, 0},
            {1, 1, 0, 0}
        };

        List<int[]> path = bfs(grid);

        if (path != null) {
            System.out.println("Shortest path from (0,0) to (" + (grid.length-1) + ","
+ (grid.length-1) + ") allowing 8 directions:");
            printPath(path);
            System.out.println("Path length: " + (path.size() - 1));
        } else {
            System.out.println("No path exists!");
        }
    }
}
```